ECE 2036 Exam 2

Spring 2025

Instructions: You have 1 hour, 15 minutes to complete this exam. The test is closed book and closed notes. No calculators, phones, or other electronic devices are allowed. Unless stated otherwise, justify your reasoning clearly to receive any partial credit. You must write your answer in the space provided on the exam paper itself. If space is needed for additional work, please use the backs of previous pages.

I. Fill in the blank:
10 points

Not properly freeing up memory that was dynamically allocated can cause a _____ in your program, which might eventually cause your program to run out of memory. If a double variable uses 8 bytes of memory and p is a pointer to a double, then p+4 will increment the address stored in p by _____. The keyword _____ is used inside a class to give an external function or class access to the private members of the class. Operator _____ allows the programmer to redefine the meaning of operators so they can be used on objects in a user-defined class. A diagram that shows the inheritance relationships between related base classes and derived classes is called a _____.

II. Pointers

25 points

Fill in the empty code sections below indicated by comments:

```
#include <iostream>
using namespace std;

// Write a global function here that prompts the user to input non-negative integers
// one-by-one, and ends the input when a negative integer is entered, your function
// should return 'void' but report the number of integers entered and the sum of the
// input values back to the calling code using pass by reference implemented with pointers
```

```
int main()
{
  int numInputs, sum;

// Call your global function here using pointers to pass numInputs and sum by reference


cout << "The sum of the " << numInputs << " numbers entered is " << sum << endl;

} // end of main()
```

III. Class composition with constructors and destructors

20 points

On the next page, show the output of the following code, where TimeDate objects contain a Date object:

```
#include <iostream>
using namespace std;

class Date {
public:
  Date(int m = 1, int d = 1, int y = 1900):month{m}, year{y}, day{d} {
    cout << "New date object: " << month << "/" << day << "/" << year << endl;
  }
  ~Date() {
    cout << "Destroying date object: " << month << "/" << day << "/" << year << endl;
  }
  int getMonth() const {return month;}
  int getDay() const {return day;}
  int getYear() const {return year;}

private:
   int month; // 1-12 (January-December)
   int day; // 1-31 based on month
   int year; // any year
};

class TimeDate {
public:
  TimeDate(int h, int m, int s, Date & d):hour{h}, minute{m}, second{s}, date{d} {
    cout << "New time/date object: " << date.getMonth() << "/" << date.getDay() << "/"
         << date.getYear() << " " << hour << ":" << minute << ":" << second << endl;
  }
  ~TimeDate() {
    cout << "Destroying time/date object: " << date.getMonth() << "/" << date.getDay()
         << "/" << date.getYear() << " " << hour << ":" << minute << ":" << second << endl;
  }
  int getHour() const {return hour;}
  int getMinute() const {return minute;}
  int getSecond() const {return second;}
  Date & getDate() {return date;}

private:
  int hour; // 0-23
  int minute; // 0-59
  int second; // 0-59
  Date date;
};

int main() {
  Date d1{5};
  Date d2{3, 26, 2025};
  TimeDate td1{12, 20, 15, d1};
  TimeDate td2{14, 30, 45, d2};

  cout << "\nTime and date of td2 is: " << td2.getDate().getMonth() << "/"
       << td2.getDate().getDay() << "/" << td2.getDate().getYear() << " " << td2.getHour()
       << ":" << td2.getMinute() << ":" << td2.getSecond() << endl << endl;
}
```

IV. Inheritance

20 points

Show the output of the following code, which has base classes and derived classes:

```
#include <iostream>
using namespace std;

class Base {
  friend int main();
public:
  void calc(int n) {
    n *= 2;
    m = n;
    k++;
  }
protected:
  static int k;
  int m;
};

class DerivedAndBase : public Base {
protected:
  int n;
};

class Derived : public DerivedAndBase {
public:
  int calc(int & n) {
    m = n * 2;
    k++;
    return m;
  }
};

void calc(int & n) {
  n *= 2;
}

int Base::k = 0;

int main() {
  int n = 1;
  calc(n);
  cout << n << endl;
  Base b;
  DerivedAndBase db;
  Derived d;
  b.calc(n);
  cout << n << endl;
  db.calc(n);
  cout << n << endl;
  int m = d.calc(n);
  cout << m << endl;
  cout << Base::k << endl;
}
```

V. Operator overloading

25 points

Given a complex number class with the below interface (Complex.h), write functions to overload the '=' and '==' operators to perform assignment of one complex number to another and to check if two complex numbers are equal. You can implement the functions either as member functions or as non-member functions but you must state which option you are implementing. Note that assignment would work for this class without overloading '=', but you should still implement a full assignment operation where real and imaginary parts are explicitly copied from one complex number to another. Lastly, check for and avoid self assignment in your '=' function, i.e. for the statement 'c1 = c2;', if c1 and c2 are equal, do not assign anything to the data members of c1.

```
class Complex
{
public:
  double getReal() const;
  void setReal(double);
  double getImag() const;
  void setImag(double);
  // add your function prototypes here, if necessary



private:
  double real;
  double imag;
};

// add your function implementations here
```